Satpuda Core App Flow Audit

Read-only source audit and relationship map - generated 2026-05-03

Scope: Sales, Purchase, Returns, Ledger, Payments, History, Customers, Inventory, Settings, and Dashboard. No application source code was modified. This PDF is a generated documentation artifact only.

Executive Summary

- The app is organized around two accounting ledgers: customer-side sales and supplier-side purchases.
- Sales writes flow through sales, sales_items, medicines stock, and customer balance recalculation.
- Purchase writes flow through purchases, purchase_items, medicines stock, and supplier balance recalculation.
- Returns and standalone payments do not directly replace the original transaction; they add separate transaction rows and then recalculate balances.
- Inventory is shared by both sides: purchases increase stock, sales decrease stock, sales returns restore stock, and purchase returns reduce stock.
- Ledgers rebuild running balances from raw transactions instead of only trusting summary columns.

Main Navigation Map

Nav entry	Primary class/function	Role
Home	ui.shared.home_page.build_home	Dashboard summaries, low stock, due lists, recent bills.
Sales	ui.billing.BillingPage	Create bill, sell medicines, decrease stock, update customer balance.
Purchase	ui.purchase.PurchasePage	Create purchase, add medicines/stock, update supplier balance.
Inventory	ui.inventory.InventoryPage	View/edit medicines, stock, expiry, location, sales/purchase history.
Sales History	ui.sales.SalesHistoryPage	List bills, view, edit, print, delete, export.
Purchase History	ui.purchase.PurchaseHistoryPage	List purchases, edit, delete, export.
Customers	ui.shared.CustomersPage	View customer balances and per-customer sales history.
Returns	SalesReturnPage / PurchaseReturnPage	Record sales returns and purchase returns.
Settings	SettingsPage tabs	Pharmacy, doctors, suppliers, payments, ledgers, import, database/export, shelves, thresholds.

Core Data Model

Area	Tables	Meaning
Sales	customers, sales, sales_items, customer_payments, sales_returns, sales_return_items	Customer account, bill headers/items, standalone payments, customer returns.
Purchase	suppliers, purchases, purchase_items, supplier_payments, purchase_returns, purchase_return_items	Supplier account, purchase headers/items, supplier payments, supplier returns.
Inventory	medicines, medicines_master, racks, sections, boxes, shelf_settings	Current stock, master medicine lookup, shelf/location management.
Setup	pharmacy_profile, doctors, settings	Bill header/profile, scheduled medicine doctor data, thresholds and app settings.

High-Level Relationship Diagram

```
Customer side
-------------
```

```
Customers 1 -> many Sales 1 -> many Sales Items -> Medicines
Customers 1 -> many Customer Payments
Sales     1 -> many Sales Returns 1 -> many Sales Return Items -> Medicines

Supplier side
-------------
Suppliers 1 -> many Purchases 1 -> many Purchase Items -> Medicines
Suppliers 1 -> many Supplier Payments
Purchases 1 -> many Purchase Returns 1 -> many Purchase Return Items -> Medicines

Inventory and setup
-------------------
Medicines optionally point to shelf locations stored as location text.
Racks -> Sections -> Boxes define valid location strings.
Pharmacy Profile is read before billing and bill preview.
Doctors are used by scheduled medicine sales.
```

Sales Flow

Create Bill

```
BillingPage
  -> get_or_create_customer()
  -> save_new_bill()
       insert sales header
       insert sales_items rows
       update medicines.stock_qty = stock_qty - sold_qty
       upsert doctor when provided
       recalculate_customer_due()
            customers.total_due / total_credit updated
            sales.account_cleared updated
  -> show_bill_preview()
```

Step	Primary files	Tables affected
Entry UI	ui/billing/billing.py, ui/billing/billing_form.py	Reads customers, medicines, doctors, pharmacy_profile, shelf_settings.
Save	core/billing_service.py	Writes sales, sales_items; updates medicines and doctors.
Balance	core/customer_service.py	Rebuilds customer total_due/total_credit from sales, customer_payments, sales_returns.
Preview	widgets/bill_preview.py	Reads pharmacy_profile, sales, customers, sales_items, medicines.

Sales History, Edit, Delete, Print

```
SalesHistoryPage
  -> reads sales + customers
  -> joins sales_items + medicines for schedule filtering and profit
  -> View: reads sales_items + medicines
  -> Print: bill_preview reads bill/header/items/profile
  -> Edit: opens widgets.bill_edit.BillEditPage -> update_existing_bill()
  -> Delete: restores stock, deletes sales_items and sales, recalculates customer due
```

Connection note: Sales History imports the edit page from widgets.bill_edit, not ui.billing.bill_edit. That live edit path should be kept in sync with the billing edit copy.

Customer Payment and Sales Return

```
CustomerPaymentTab
  -> insert/update/delete customer_payments
  -> recalculate_customer_due()

SalesReturnPage
  -> choose original sale and sale items
  -> insert sales_returns and sales_return_items
  -> update medicines.stock_qty = stock_qty + returned_qty
  -> recalculate_customer_due()
```

Purchase Flow

Create Purchase

```
PurchasePage
```

```
  -> get_or_create_supplier()
  -> get_or_create_medicine() for each item
  -> PurchaseCalculator.calculate()
  -> save_purchase()
       insert purchases header
       insert purchase_items rows
       update medicines.stock_qty = stock_qty + qty/free_qty
       set amount_paid_at_entry snapshot
       recalculate_supplier_due()
            suppliers.total_due / total_credit updated
            purchases.account_cleared updated
```

Step	Primary files	Tables affected
Entry UI	ui/purchase/purchase.py, ui/purchase/purchase_form.py	Reads suppliers, medicines_master, medicines.
Save	core/purchase_service.py	Writes suppliers, medicines, purchases, purchase_items; updates stock.
Calculation	core/purchase_calculator.py	Computes subtotal, GST, discounts, due/credit.
Balance	core/purchase_service.py	Rebuilds suppliers.total_due/total_credit from purchases, supplier_payments, purchase_returns.

Purchase History, Edit, Delete

```
PurchaseHistoryPage
  -> reads purchases + suppliers + purchase_items
  -> includes purchase_returns refund totals
  -> Edit: opens PurchasePage and loads existing purchase_items
           update_purchase() reverses old stock, replaces items, recalculates supplier
  -> Delete: validates stock, reverses purchase stock, deletes purchase/items, recalculates supplier
```

Supplier Payment and Purchase Return

```
PaymentTab
  -> insert/delete supplier_payments
  -> recalculate_supplier_due()

PurchaseReturnPage
  -> choose original purchase and purchase items
  -> insert purchase_returns and purchase_return_items
  -> update medicines.stock_qty = stock_qty - returned_qty_or_tablets
  -> recalculate_supplier_due()
```

Ledger Flow

The ledger tab is a reporting/reconstruction layer. It does not write balances. It collects raw transactions in date order and calculates a running balance on screen.

Ledger	Raw sources	Balance logic
Customer Ledger	sales, sales_returns, customer_payments, customers	Running = sales total - bill paid - returns - standalone payments. Positive means due, negative means credit.
Supplier Ledger	purchases, purchase_returns, supplier_payments, suppliers	Running = purchase total - entry paid - returns - supplier payments. Positive means supplier due, negative means supplier credit.

Customers and Suppliers

Customers

- CustomersPage reads customer summary rows through get_all_customers().
- Customer balances are cached in customers.total_due and customers.total_credit.
- Recalculate Due uses recalculate_customer_due(), which rebuilds from sales, customer_payments, and sales_returns.
- Customer sales history reads sales for the selected customer.

Suppliers

- SuppliersTab reads suppliers.total_due and suppliers.total_credit as the current supplier balance.
- Recalculate Balance uses recalculate_supplier_due(), which rebuilds from purchases, supplier_payments, and purchase_returns.
- Supplier delete is blocked when purchase history exists.

Inventory Flow

```
Purchase save          -> medicines.stock_qty increases
Purchase edit/delete   -> old purchase stock is reversed before replacement/deletion
Purchase return        -> medicines.stock_qty decreases
Sales bill             -> medicines.stock_qty decreases
Sales edit/delete      -> old sold stock is restored before replacement/deletion
Sales return           -> medicines.stock_qty increases
Inventory edit         -> direct manual update of medicine fields and stock
Shelf Management       -> updates medicines.location strings
```

InventoryPage reads medicines for stock, expiry, type, unit, MRP/rate, manufacturer, schedule, and optional location. Inventory detail dialogs connect each medicine back to its purchase_items and sales_items history.

Page Connection Matrix

Page/tab	Reads from	Writes to / calls
Home Dashboard	sales, purchases, medicines, customers, suppliers	No writes; navigation shortcuts and exports.
Billing	customers, medicines, doctors, pharmacy_profile, shelf_settings	save_new_bill -> sales, sales_items, medicines, doctors, customers.
Sales History	sales, customers, sales_items, medicines, customer_payments, sales_returns	Edit/print/delete actions; delete restores stock and recalculates customer.
Customer Payment	customers, customer_payments	customer_payments; recalculate_customer_due.
Sales Return	sales, sales_items, medicines, sales_returns	sales_returns/items; medicines stock increase; recalculate_customer_due.
Purchase	suppliers, medicines, medicines_master	save_purchase -> purchases, purchase_items, medicines, suppliers.
Purchase History	purchases, suppliers, purchase_items, purchase_returns	Edit/delete; reverse stock and recalculate supplier.
Supplier Payment	suppliers, supplier_payments	supplier_payments; recalculate_supplier_due.
Purchase Return	purchases, purchase_items, medicines, purchase_returns	purchase_returns/items; medicines stock decrease; recalculate_supplier_due.
Ledger	sales/purchases, returns, payments, customers/suppliers	No writes; calculates running balance display.
Customers	customers, sales	Manual recalculation of customer balances.
Inventory	medicines, settings, shelf_settings, sales_items, purchase_items	Medicine edit/delete; location display.
Shelf Management	racks, sections, boxes, medicines	Shelf CRUD; updates medicines.location.

Connection Risks Found During Read-Only Audit

Risk	Where	Impact
Fresh DB init expects optional accounting tables before they are created.	core/db_setup.py one-time migration references supplier_payments, purchase_returns, purchase_return_items, sales_returns, sales_return_items.	A brand-new in-memory init printed: no such table: supplier_payments. Existing live DB already has these tables.
Sales edit live path has positional-index mismatch in widgets.bill_edit.	ui/sales/sales_history_actions.py imports widgets.bill_edit; widgets/bill_edit.py uses SELECT si.* and reads item[7] as name.	Edit Bill can load sales item fields incorrectly. The ui/billing copy has one later fix but still uses fragile si.* positions for other fields.
Dashboard due summaries read transaction snapshots.	ui/shared/home_page.py reads sales.total_due and purchases.total_due for due widgets.	Can differ from authoritative customers.total_due and suppliers.total_due after payments/returns.

Risk	Where	Impact
Database purchase export reads old purchase fields.	ui/settings/settings_tabs/database_tab.py exports amount_paid, due_amount, total_due.	Can differ from amount_paid_at_entry and supplier balance model used by Purchase History.
Legacy sales rebuild omits discount_pct.	core/db_setup.py _rebuild_sales_table.	Only triggered for old phone_pay_paid schema, but would drop discount_pct if it runs.

Recommended Flow Rules

- Use customers.total_due / total_credit as the current customer account balance.
- Use suppliers.total_due / total_credit as the current supplier account balance.
- Use sales and sales_items as immutable bill history except during edit/delete flows that restore stock first.
- Use purchases.amount_paid_at_entry for purchase entry payment, because supplier payments are tracked separately.
- Use returns tables as independent transactions and recalculate balances after each return.
- Avoid SELECT * or table.* where positional indexing is used; explicit column lists keep pages stable after migrations.

Files Reviewed

main.py; core/db_setup.py; core/billing_service.py; core/purchase_service.py; core/customer_service.py; core/purchase_calculator.py; core/calc_engine.py; ui/billing/*; widgets/bill_edit.py; widgets/bill_preview.py; ui/sales/*; ui/purchase/*; ui/returns/*; ui/settings/settings.py and settings_tabs/*; ui/inventory/*; ui/shared/customers.py; ui/shared/home_page.py; ui/shared/shelf_management.py; ui/shared/import_purchases.py; ui/shared/import_from_mobile.py.

Verification method: source tracing with ripgrep and direct SQLite schema introspection against veterinary.db. No source files were edited and no app data was modified.